

One hour, offline, with two people in the room

What the ceremony is for, what to buy before it, what happens in the hour, and how we prove afterwards that it actually took — step by step, so the day itself is unremarkable.

DURATION ~60 min	PEOPLE Two	PHYSICAL ITEMS Four
REPEATS Never		

01 · THE POINT

What the hour is actually for

Our units install firmware only if it carries a valid signature. That mechanism works and has been proven on hardware. What it currently signs with is a development key generated per checkout, sitting in a working directory — which means the update path is, today, exactly as trustworthy as a laptop.

Whoever holds this key can put software on every unit we ever ship. It has to be harder to steal than any single connected computer at Joral.

That is the whole rationale, and it is why the answer is an offline ceremony rather than a key on the build server. The hour produces two keypairs and four physical items, and it never has to happen again unless a key is compromised.

The ceremony generates an **active** key and a **standby**. Both public halves are baked into every image from the very first unit; the standby signs nothing until the day it has to. That is what turns a future key compromise into a signed update instead of a recall — and it is the reason this cannot be done after shipping. A second trust anchor added later reaches units only by physical access.

Bring these to the room

One of these has a delivery time. The rest is an afternoon of preparation by the engineer, done before the meeting.

☐ **Four identical USB sticks**

Two labelled sets of two. This is the only thing to order, and ordering it costs nothing if the meeting slips.

☐ **A laptop that can boot from USB**

Any machine. Nothing is installed on it and nothing survives the session.

☐ **A live-Linux USB, checksum verified**

Prepared in advance on a normal machine. The session runs entirely in memory.

☐ **A transfer stick with the ceremony script**

Ordinary and unencrypted — it carries the script in, and public certificates out. Never a private key.

☐ **An agreed passphrase, decided before the keyboard**

One passphrase, both holders know it. Consider two sealed paper envelopes, one per key set.

☐ **Labels and a pen**

The minute is signed on paper and filed. An assessor asking how we hold the signing key is asking for that sheet.

ONE THING TO KNOW BEFORE ORDERING

The policy says “hardware tokens or encrypted media”. As the procedure is actually written, the keys come out as encrypted files, so the practical purchase is **encrypted USB sticks** — a PIV token or HSM would need an import step this procedure does not have. If a token is wanted later, the documented upgrade is a dedicated signing station holding the *same* keys, which changes how we sign and never touches a shipped unit.

03 · THE HOUR

The ceremony, in order

Present: you and the named engineer, for the whole session. Both of you become passphrase holders — two people, so that a departure or an accident never strands the fleet.

1 Set the room

Ethernet unplugged, Wi-Fi hardware disabled, Bluetooth off. The script refuses to run if it sees a network route, but that is a seatbelt; the procedure is what makes it true.

2 Boot the live USB, and check the clock

A live session can start with the wrong date, and the certificates take their start date from it. Set it from a watch or a phone — not from a network.

3 Run the script, generating into memory

It creates both keypairs and their certificates, assembles the trust bundle that ships in every image, and prints a pre-filled minute. It asks for the passphrase four times: two keys, each confirmed.

```
mkdir -p /run/ceremony
sh fw-key-ceremony.sh /run/ceremony
```

4 Prove the passphrase before destroying anything

Open both keys with it. Each must prompt, accept and succeed.

```
openssl pkey -in /run/ceremony/private/joral-fw-2026-a.ke  
openssl pkey -in /run/ceremony/private/joral-fw-2026-b.ke
```

5 Write the four media — and read each one back

Both keys onto every stick, encrypted. Then open each copy *from the stick* before moving on. Label as you go: set one for the safe, set two for off-site.

6 Take the public halves out on the transfer stick

Three certificate files. They are meant to be public; they go into the repository and into every image.

7 Sign the minute

The script fills in the date and both certificate fingerprints. You add the two names and the four media serials, and both of you sign it. It goes into the technical file.

8 Shred the working copies and power off

The session lived in memory; powering off ends it. The shred is belt and braces.

9 Separate the two sets the same day

Safe and off-site. Both sets in one building overnight is precisely the failure the second set exists to prevent.

THE ONE PLACE THE ORDER MATTERS

Steps 4 and 5 come before step 8. A passphrase both people believed they knew, typed once and never tested, is the single mistake in this procedure that cannot be walked back once the working copies are gone — and the cost of catching it early is thirty seconds.

What changes in the build, and how little of it is code

The switchover is a file arriving in the repository. Nothing is configured, no flag is set, and nothing can be forgotten: the build looks for the production certificates, and behaves differently depending on whether they are there.

Now	Development The build stages a per-checkout development trust store and says NOT FOR SHIPMENT in the log. Bench units accept those packages; a customer unit never would.
After	Production The build stages the real trust store and says so. Packaging a release now <i>stops</i> and prints offline-signing instructions rather than signing anything itself.

From then on, each release costs about fifteen minutes: the build produces the package and prints its fingerprint, the engineer carries it to the offline laptop, verifies the fingerprint, signs, carries the signature back, and the build verifies the result against the trust store the image actually ships before calling it a release. One line goes into a signing log — date, version, fingerprint, who signed, which media.

Two small pieces of housekeeping belong to the same week: the signing log itself does not exist yet, and the certificate fingerprints need to go into the compliance fact sheets for both products.

Proving it took

A build log line is a claim. Three checks turn it into evidence, and the third is the one that matters:

- **The staged bundle holds exactly two certificates**, both ours, and no development certificate.
- **Read it out of the packed image**, not the tree that fed it. A staging directory can disagree with the artifact, which is exactly the class of defect this programme keeps finding.
- **Keep one development-signed package and try to install it on a production unit.** It must be *refused*. A unit that accepts both would mean the development certificate survived into the bundle, and nothing in the build would have said so.

Ten minutes on the bench, and it is worth adding as a leg of the next session. Then the positive: install the production-signed package, watch the slot swap, confirm the health

If something goes wrong

The passphrase is wrong, before the shred

Re-run the ceremony. Nothing has shipped, so the cost is an hour and some embarrassment. This is why the whole procedure happens before first delivery rather than after.

A stick fails, later

There are four copies for this reason. Replace it, record it in the signing log, change the passphrase at the next signing.

A key or its passphrase is compromised, after units ship

Rotate to the standby key and push a signed update. This works only because both public halves shipped in every image from the first unit — which is the entire reason the ceremony produces two keys rather than one.

ONE GAP WORTH CLOSING WHILE WE ARE HERE

Nothing in the build *mechanically* refuses to ship a development-keyed image today. It warns, and the discipline has held, but the release gates do not look at the trust store. The check is about twenty lines of script, and the window it guards is open right now — after the ceremony the question stops arising, because the production certificates are simply there.

What the hour buys

It closes the second of the doors that shut when the first customer unit ships. After it, the update mechanism is not merely working but *ours*: every unit in the field will accept firmware from two keys that exist on four items in two buildings, held by two people, with a written procedure for the day one of them goes wrong.

Before it, the honest description of our update path is that it trusts a key generated by a build script. That is a perfectly good answer for a bench unit and not an answer we can give a customer, an auditor, or ourselves.

This page is the readable version of the operational runbook kept with the engineering documents, which carries the exact commands, the post-ceremony build steps and the verification checks. Where the two differ, the runbook and the signed policy are authoritative.